Hinge-chain networks

A deep ReLU network of width two that draws any piecewise-linear function, one kink per layer

A ReLU network computes a continuous piecewise-linear function: straight pieces that meet at kinks. These notes run the other way. Take any continuous piecewise-linear function f with k kinks; we build a ReLU network whose output is exactly f, with k hidden layers of two neurons, so that every layer adds one kink. In each layer a *hinge* neuron makes the kink and a *running-sum* neuron collects the kinks made so far. The hinges form one chain and the running sums another, so we call it a *hinge-chain network*.

We build the network one part at a time, each with a picture first and its formula second, on one running example, the zigzag of Figure 1. Every weight and bias can then be read off f, and only $2k + 1$ of them are free: the *knobs* of the interactive page *Hinge-chain networks*, where you can turn them by hand or let the page set them (Section 9). The slides of the lecture use the same notation, the same example and the same colours: hinges in vermillion, running sums in green, knobs in violet.

The main text states results as claims; the proofs are in Appendix A, and the solutions of the exercises in Appendix B. You only need to know what a ReLU neuron is.

§1 The goal: draw the zigzag

Our running example is the zigzag of Figure 1. It starts at the point $(-2, -1)$ and joins the points $(-1, 1)$, $(0, 0)$, $(1, 2)$ and $(2, -1)$ by straight pieces. It is also the target that the page starts with.

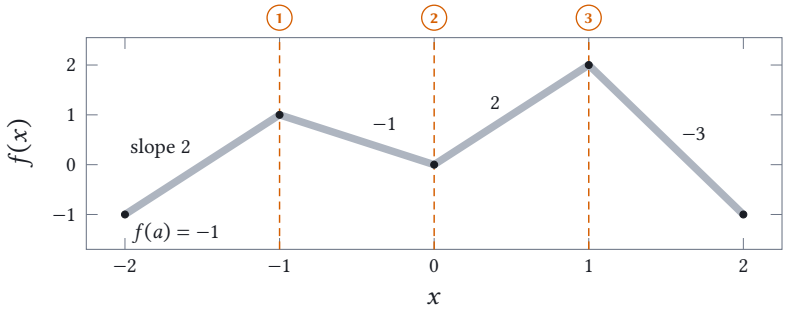


Figure 1: The zigzag, the running example of these notes, the slides and the page. The dashed lines are its kinks t_1, t_2, t_3, numbered as on the page; the numbers on the pieces are their slopes.

A function like this is *continuous piecewise linear*: straight pieces that meet at *kinks*, the points where the slope changes. Three kinds of numbers describe it completely: where it starts, how steep each piece is, and where the kinks are. Start at the height $f(a)$, walk to the right along the first slope, turn to the next slope at each kink, and you have drawn f.

In symbols, $f : [a, b] \to \mathbb{R}$ is continuous and piecewise linear with $k \geq 1$ kinks

$$a < t_1 < t_2 < \cdots < t_k < b.$$

We put $t_0 = a$ and $t_{k+1} = b$, so that the pieces are the intervals $[t_i, t_{i+1}]$ for $i = 0, \ldots, k$, and we write s_i

for the slope of f on the piece $[t_i, t_{i+1}]$. For the zigzag, $[a, b] = [-2, 2]$, $k = 3$ and

$$f(a) = -1, \qquad (t_1, t_2, t_3) = (-1, 0, 1), \qquad (s_0, s_1, s_2, s_3) = (2, -1, 2, -3).$$

The goal. A ReLU network with one input x and one output y such that $y(x) = f(x)$ for every $x \in [a, b]$, exactly, with every weight and bias read off f.

§2 The hinge

Everything is built from one part, the ReLU. It acts like a floor: it passes its input on when the input is positive and cuts off whatever goes below zero (Figure 2a). Shift its input, and the corner of the floor moves along the x-axis to a point t: you get a *hinge*, flat up to t and rising with slope 1 after it (Figure 2b).

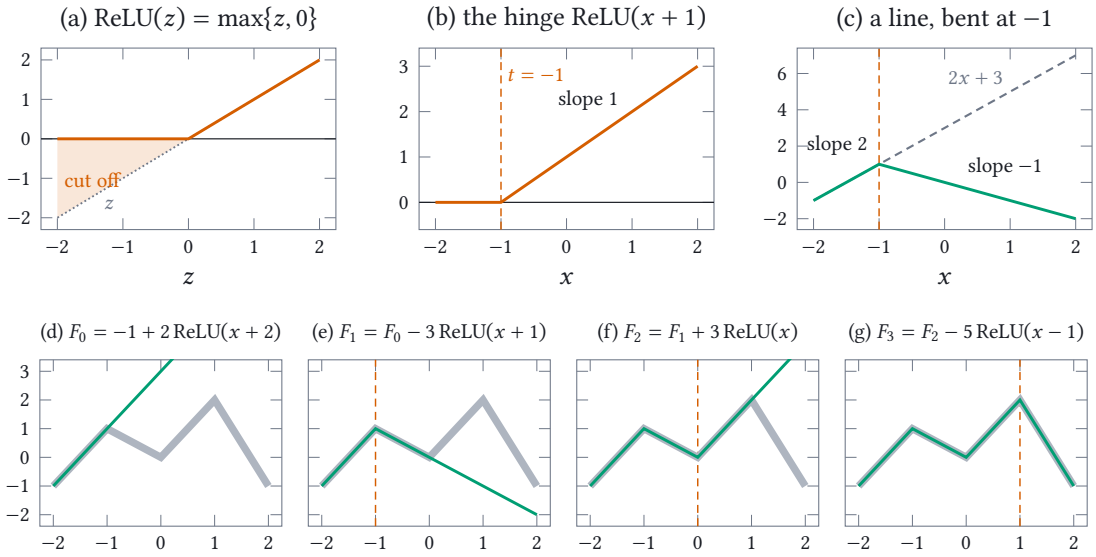


Figure 2: The hinge and the hinge form. (a) The ReLU is a floor: whatever goes below zero is cut off (shaded). (b) The hinge $\mathrm{ReLU}(x + 1)$ has its kink at $t = -1$. (c) Adding $-3\,\mathrm{ReLU}(x + 1)$ to the line $2x + 3$ (dashed) bends it at -1 by -3: the slope goes from 2 to -1, the first kink of the zigzag. (d) to (g) The zigzag (grey), hinge by hinge (green). Each panel adds the hinge in its title, which bends the curve at the dashed line; (d) is the first piece, continued to the right end.

In symbols, $\mathrm{ReLU}(z) = \max\{z, 0\}$, and feeding it $x - t$ gives the hinge

$$\mathrm{ReLU}(x - t) = \begin{cases} 0 & x \le t, \\ x - t & x \ge t, \end{cases}$$

with a single kink, at t. A weight after the ReLU scales the hinge: $\Delta\ \mathrm{ReLU}(x - t)$ is flat up to t and has slope Δ after it. So the bias inside the ReLU, $-t$, says *where* the hinge bends, and the weight after it says *by how much*. Now add such a hinge to a line. Left of t nothing changes; right of t the slope grows by Δ. Adding $\Delta\ \mathrm{ReLU}(x - t)$ to a line bends it at t by Δ, upwards if $\Delta > 0$ and downwards if $\Delta < 0$ (Figure 2c).

§3 The hinge form

Bend a line once at every kink, and you get any piecewise-linear function. The bottom row of Figure 2 builds the zigzag this way, one hinge at a time: a line with the first slope, then a bend at each kink.

At the kink t_i the slope changes from s_{i-1} to s_i, so the hinge that we add there gets the weight

$$\Delta_i = s_i - s_{i-1} \qquad (i = 1, \dots, k),$$

the *slope change* at kink i. The first piece needs a slope of its own, s_0, and a hinge provides that too: the hinge at the left end, $\mathrm{ReLU}(x - a)$, is the straight line $x - a$ on all of $[a, b]$. With $t_0 = a$ it is hinge number 0, and its weight $\Delta_0 = s_0$ is the first slope.

Claim 1 (Hinge form). *For every $x \in [a, b]$,*

$$f(x) = f(a) + \sum_{i=0}^{k} \Delta_i \ \mathrm{ReLU}(x - t_i). \tag{1}$$

Both sides equal $f(a)$ at $x = a$, where every hinge is zero, and on each piece $[t_i, t_{i+1}]$ both have the slope $\Delta_0 + \Delta_1 + \cdots + \Delta_i = s_i$, so they agree everywhere (Appendix A).

For the zigzag, the slopes $(2, -1, 2, -3)$ give the slope changes

$$(\Delta_0, \Delta_1, \Delta_2, \Delta_3) = (2, -3, 3, -5).$$

The line starts with slope 2, bends down by 3 at -1, up by 3 at 0 and down by 5 at 1, so

$$f(x) = -1 + 2\,\mathrm{ReLU}(x + 2) - 3\,\mathrm{ReLU}(x + 1) + 3\,\mathrm{ReLU}(x) - 5\,\mathrm{ReLU}(x - 1).$$

Panels (d) to (g) of Figure 2 are the *partial sums*

$$F_\ell(x) = f(a) + \sum_{i=0}^{\ell} \Delta_i \ \mathrm{ReLU}(x - t_i) \qquad (\ell = -1, 0, \dots, k), \tag{2}$$

which keep the first ℓ kinks of f and drop the rest. The empty sum $F_{-1} = f(a)$ is a constant, each step adds one hinge, $F_\ell = F_{\ell-1} + \Delta_\ell\,\mathrm{ReLU}(x - t_\ell)$, and the last step gives $F_k = f$. In between, F_ℓ is f up to the kink $t_{\ell+1}$ and goes on straight after it, with the slope s_ℓ of the last piece it has seen. The partial sums will come back as the running sums of the network.

Equation (1) is itself a network, with one hidden layer of $k + 1$ hinge neurons $\mathrm{ReLU}(x - t_i)$ and an output that weights them by Δ_i and adds $f(a)$. This hinge-fan network has its own note, *A piecewise-linear function as a shallow ReLU network*, and its own page, *Hinge-fan networks*. Here we build f the other way: deep and narrow, one kink per layer.

§4 One kink per layer

Why go deep? In a deep network every layer can have a job that you can name, and here the job is simple: layer ℓ adds kink ℓ. The curve then grows one kink per layer, and after layer ℓ the network has drawn f up to its ℓth kink. How many neurons does a layer need? One is too few.

Claim 2 (Width one is too narrow). *A ReLU network with a single neuron in every hidden layer computes a monotone function of x: it can only go up, or only go down.*

The reason is that a layer with a single neuron maps its input u to $\mathrm{ReLU}(wu + c)$, which rises with u when $w \geq 0$ and falls when $w \leq 0$, and a chain of maps that each rise or fall again rises or falls (Exercise 4 writes this out). The zigzag rises, falls, rises and falls, so no such network draws it.

Two neurons per layer are enough, if each does its own job. Picture two lanes that run side by side through the layers, as in Figure 3.

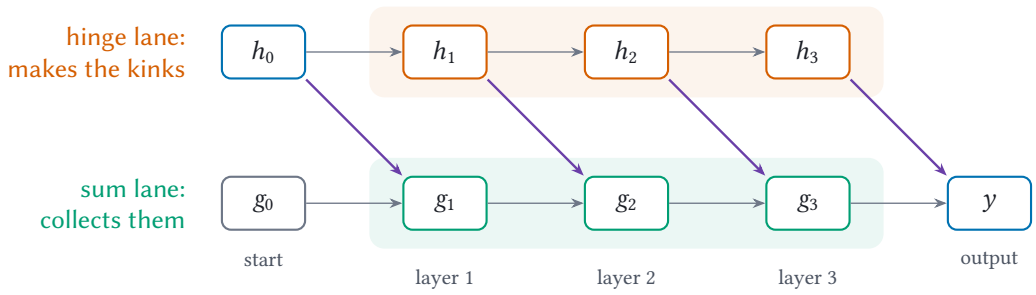


Figure 3: The plan, for $k = 3$ layers of two neurons. The hinge lane (top) makes one kink per layer, the sum lane (bottom) collects them, and the output y reads off the total. A grey arrow passes a value on unchanged; a violet arrow carries a hinge into the sum lane with a weight, its slope change.

- The *hinge lane* makes the kinks. Layer ℓ holds the *hinge neuron* h_ℓ, whose kink sits at t_ℓ.
- The *sum lane* collects them. Layer ℓ holds the *running-sum neuron* g_ℓ, which adds a hinge, weighted by its slope change, to the total so far.

The hinge lane never reads the sum lane. The sum lane reads one hinge per layer, along the diagonal arrows, and at the end the output y reads off the total. The next two sections build the two lanes, one at a time.

§5 The hinge lane

The hinge lane starts from the input, measured from the left end, $h_0 = x - a$: a ramp that is zero at $x = a$ and rises with slope 1, so on $[a, b]$ it is the hinge $\mathrm{ReLU}(x - t_0)$. For the zigzag, $h_0 = x + 2$. It is a starting value, computed from x without a ReLU.

Each hinge neuron moves the previous kink to the right: it lowers the previous hinge and cuts at zero. Figure 4a shows why this moves the kink. Lowering a hinge by d makes it negative on a stretch of length d beyond its kink, and the floor of the ReLU cuts that stretch away. What is left is the same hinge, with its kink d further to the right.

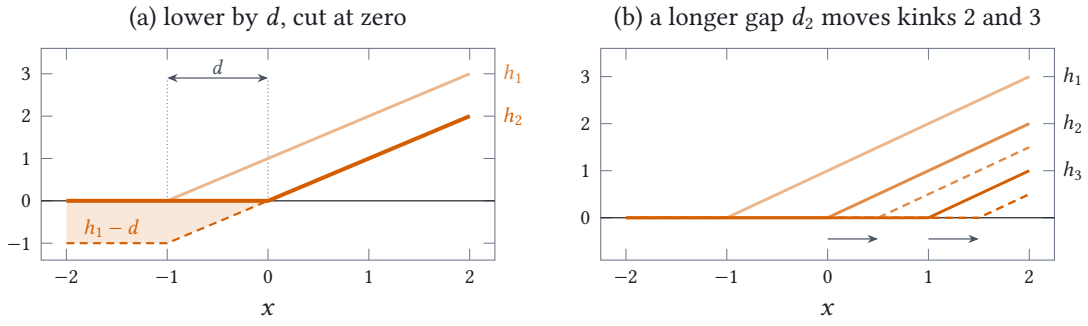


Figure 4: The hinge lane. (a) A hinge neuron lowers the previous hinge by its gap d (dashed), and the ReLU cuts off what falls below zero (shaded). What is left is the same hinge, moved right by d: here $h_1 = \mathrm{ReLU}(x + 1)$ and $d = 1$ give $h_2 = \mathrm{ReLU}(h_1 - d) = \mathrm{ReLU}(x)$. (b) The chain. In the zigzag network (solid), lengthening the gap d_2 from 1 to 1.5 (dashed) moves kinks 2 and 3 to the right by 0.5, and kink 1 stays.

In symbols, hinge neuron ℓ lowers the previous hinge by a number $d_\ell \geq 0$ and cuts at zero,

$$h_\ell = \mathrm{ReLU}\big(h_{\ell-1} - d_\ell\big) \qquad (\ell = 1, \dots, k), \tag{3}$$

and the picture is the following claim.

Claim 3 (Shift). *For every $u \in \mathbb{R}$ and every $d \geq 0$,* $\mathrm{ReLU}\big(\mathrm{ReLU}(u) - d\big) = \mathrm{ReLU}(u - d)$.

Take $u = x - t_{\ell-1}$: a hinge with its kink at $t_{\ell-1}$, lowered by d_ℓ and cut, is a hinge with its kink at $t_{\ell-1} + d_\ell$. So we choose each d_ℓ as the *gap* from one kink to the next,

$$d_\ell = t_\ell - t_{\ell-1} \qquad (\ell = 1, \dots, k;\ \text{so } d_1 = t_1 - a),$$

and the gaps are the steps along x from one kink to the next.

Claim 4 (The hinge lane). *With these gaps, $h_\ell(x) = \mathrm{ReLU}(x - t_\ell)$ for every $x \in [a, b]$ and $\ell = 0, \dots, k$.*

For the zigzag, the kinks $-1, 0, 1$ are one step apart, and the first sits one step after $a = -2$, so $d = (1, 1, 1)$ and

$$h_1 = \mathrm{ReLU}(x + 1), \qquad h_2 = \mathrm{ReLU}(x), \qquad h_3 = \mathrm{ReLU}(x - 1).$$

The kinks form a chain, $t_\ell = a + d_1 + d_2 + \cdots + d_\ell$. Lengthen one gap, and its kink moves to the right together with every kink after it, while the kinks before it stay (Figure 4b); this is what you see on the page when you drag a kink. The gaps must never be negative. A hinge "lowered" by a negative amount goes up, so it never reaches the floor, its ReLU never cuts, and its kink stays where it was (Exercise 3). A gap of zero is allowed: it puts two kinks on top of each other.

§6 The sum lane

The sum lane keeps a running total of the hinge form: it starts from the start value $f(a)$, and each layer adds one hinge, weighted by its slope change. Figure 5 shows the one trap on the way. A running total can dip below zero, and the running-sum neurons, which are ReLU neurons too, would cut it there; a constant $C \geq 0$, the *lift*, keeps it above zero.

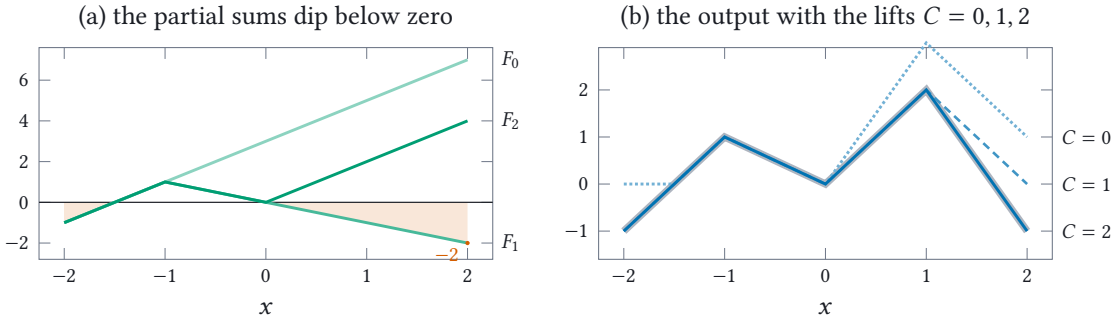


Figure 5: The sum lane needs a lift. (a) The partial sums F_0, F_1 and F_2 of the zigzag dip below zero (shaded), F_1 down to -2 at $x = 2$, so the floor of the running-sum neurons would cut them. (b) The output y of the zigzag network with the lifts $C = 0$ (dotted), $C = 1$ (dashed) and $C = 2$ (solid), against the zigzag (grey). Only $C = 2$, the least lift, draws the zigzag.

In symbols, the sum lane starts from $f(a)$ raised by the lift, and each layer adds the hinge before it, weighted by its slope change:

$$g_0 = C + f(a), \qquad g_\ell = \mathrm{ReLU}\big(g_{\ell-1} + \Delta_{\ell-1}\, h_{\ell-1}\big) \qquad (\ell = 1, \dots, k). \tag{4}$$

Like h_0, g_0 is a starting value: a constant, with no ReLU. Look at the indices: g_ℓ adds the hinge $h_{\ell-1}$ of the layer before, which is ready by the time g_ℓ is computed (the diagonal arrows of Figure 3). So the

sum lane runs one hinge behind the hinge lane. The output catches up: it adds the last hinge and takes C away again,

$$y = \Delta_k\, h_k + g_k - C. \tag{5}$$

Ignore the ReLU in (4) for a moment. Then, by Claim 4, the sum lane adds up the hinge form (1) term by term: $g_\ell - C$ is the partial sum $F_{\ell-1}$, and $y = F_{k-1} + \Delta_k \,\mathrm{ReLU}(x - t_k) = F_k = f$. In words: after layer ℓ, the running total minus C is f up to the kink t_ℓ, and a straight line after it.

The floor. The running-sum neurons are ReLU neurons too, and their floor cuts off everything below zero. A partial sum can well go negative. For the zigzag, $F_0 = 2x + 3$ starts at -1, and F_1, which equals $-x$ right of -1, ends at -2 (Figure 5a). With $C = 0$, the neuron g_1 would cut F_0 off left of -1.5, the neuron g_2 would cut F_1 off right of 0, and the network would draw something else (Figure 5b).

The lift. Think of the running total as a bank account that may never be overdrawn. We open it with a deposit C, large enough that the balance never goes below zero, and the output pays the deposit back with its $-C$. This deposit is the lift C in $g_0 = C + f(a)$. It must cover the deepest dip of every partial sum that passes through a running-sum neuron, that is, of $F_0, \dots, F_{k-1}$:

$$C + F_i(x) \geq 0 \qquad \text{for every } x \in [a, b] \text{ and } i = 0, \dots, k-1. \tag{6}$$

(The last partial sum, $F_k = f$, only reaches the output, which has no ReLU.) The page always takes the least lift,

$$C_* = \max\Big\{0,\ -\min_{0\leq i\leq k-1}\ \min_{x\in[a,b]} F_i(x)\Big\},$$

which is quick to compute: each F_i is piecewise linear, so its lowest point lies at a, at b or at a kink. For the zigzag the deepest dip is $F_1(2) = -2$, so $C = 2$. Then every running sum stays at zero or above, and $g_2 = 2 + F_1$ just touches the floor at $x = 2$.

Claim 5 (What every neuron computes). *If the lift C satisfies* (6)*, then for every $x \in [a, b]$*

$$h_\ell(x) = \mathrm{ReLU}(x - t_\ell), \qquad g_\ell(x) = C + F_{\ell-1}(x) \qquad (\ell = 0, \dots, k), \qquad \textit{and} \qquad y(x) = f(x).$$

Claim 6 (The lift cannot be dropped). *If* (6) *fails at some $x \in [a, b]$, then $y(x) > f(x)$.*

The reason for Claim 6: a ReLU never lowers a value, $\mathrm{ReLU}(v) \geq v$, so a running sum that is cut comes out too high, and the excess travels through every later layer into y (Appendix A).

How large the lift must be depends on where f sits. The note that introduced these networks, *A piecewise-linear function as a deep, narrow ReLU network*, works the same zigzag on $[0, 4]$, through the points $(0, 2)$, $(1, 4)$, $(2, 3)$, $(3, 5)$ and $(4, 2)$, with the same slope changes and gaps. There every partial sum stays at 1 or above, so no lift is needed: $C = 0$. Our zigzag is that function moved left by 2 and down by 3. Moving it down by 3 lowers every partial sum by 3, the lowest from 1 to -2, and the lift becomes $C = 2$. Figure 5b shows what a smaller lift does: with $C = 1$, the neuron g_2 is cut right of $x = 1$ and y overshoots the zigzag there by $x - 1$; with $C = 0$, the network is wrong left of -1.5 and right of 0.

§7 The knobs

Look back at the rules (3), (4) and (5), and count the numbers that the construction had to choose. Everything else in the network is a 1 or a 0:

- a hinge neuron reads the previous hinge with weight 1 and the running sum with weight 0;
- a running-sum neuron passes its own value on with weight 1 and has no bias of its own.

Figure 6 draws the zigzag network the way the page draws it: the hinges on top, the running sums below, every box showing the curve of its neuron, and a violet chip wherever the construction has a free number.

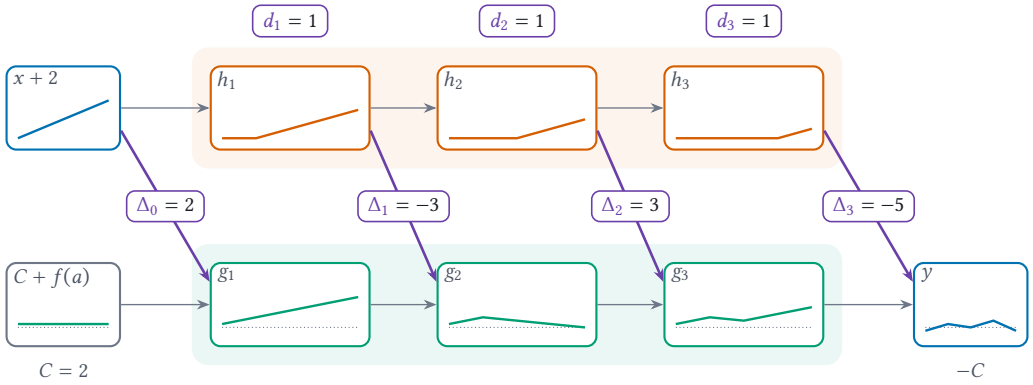


Figure 6: The zigzag network, drawn as the page draws it. Top row: the input $h_0 = x + 2$ (blue) and the hinges (vermillion). Bottom row: $g_0 = C + f(a)$ with the lift $C = 2$ written under it, the running sums (green), and the output y (blue), which takes C off again. Every box draws its neuron on $[-2, 2]$: the hinges on one scale, the running sums and y on another, with a dotted line at 0. The grey arrows have the fixed weight 1; the knobs are violet, each gap d_ℓ above its hinge and each slope change Δ_ℓ on the arrow from h_ℓ into the next running sum.

Here is the whole network in one box, with its free numbers in violet.

The hinge-chain network

Start from $h_0 = x - a$ and $g_0 = C + f(a)$. For $\ell = 1, \dots, k$,

$$h_\ell = \mathrm{ReLU}\big(h_{\ell-1} - d_\ell\big), \qquad g_\ell = \mathrm{ReLU}\big(g_{\ell-1} + \Delta_{\ell-1}\, h_{\ell-1}\big),$$

and the output is $y = \Delta_k\, h_k + g_k - C$. The knobs are the gaps $d_1, \dots, d_k$ and the slope changes $\Delta_0, \dots, \Delta_k$. The start value $f(a)$ comes from the target, and the lift C is the least that satisfies (6).

In matrix form, every layer $\ell = 1, \dots, k$ is the same map of the pair (h, g):

$$\begin{pmatrix} h_\ell \\ g_\ell \end{pmatrix} = \mathrm{ReLU}\left(\begin{pmatrix} 1 & 0 \\ \Delta_{\ell-1} & 1 \end{pmatrix} \begin{pmatrix} h_{\ell-1} \\ g_{\ell-1} \end{pmatrix} + \begin{pmatrix} -d_\ell \\ 0 \end{pmatrix} \right), \qquad y = \begin{pmatrix} \Delta_k & 1 \end{pmatrix} \begin{pmatrix} h_k \\ g_k \end{pmatrix} - C. \tag{7}$$

The free numbers are the violet ones:

- the *gaps* $d_1, \dots, d_k$, one per layer: the intercept of each hinge, which places its kink one gap after the kink before;
- the *slope changes* $\Delta_0, \dots, \Delta_k$, one per layer and one more for the output: the weight after each hinge, on the arrow that carries h_ℓ into the next running sum, or into y for Δ_k.

That makes $k + (k + 1) = 2k + 1$ *knobs*, 7 for the zigzag. They are the knobs of the page, which follows one rule: a knob appears exactly where the construction has a free number. Two more numbers enter the network, and neither is a knob. The start value $f(a)$ is read off the target, and the lift C is computed from all the other numbers, as the least lift.

For comparison, a general network of this shape has $6k + 1$ weights and biases: 2 weights and 2 biases in the first layer, 4 and 2 in every further layer, 2 and 1 at the output. The construction leaves only the $2k + 1$ knobs free, 7 of 19 for the zigzag; every other weight and bias is 0 or 1, or follows from a, $f(a)$ and C.

§8 The zigzag in full

Here is the zigzag network with every number. Its knobs are

$$d = (d_1, d_2, d_3) = (1, 1, 1), \qquad \Delta = (\Delta_0, \Delta_1, \Delta_2, \Delta_3) = (2, -3, 3, -5),$$

its start value is $f(a) = -1$ and its lift $C = 2$, so that $g_0 = 1$. The rules (3), (4) and (5) become

$$\begin{aligned} h_0 &= x + 2, & g_0 &= C + f(a) = 1, \\ h_1 &= \mathrm{ReLU}(h_0 - 1) = \mathrm{ReLU}(x + 1), & g_1 &= \mathrm{ReLU}(g_0 + 2\,h_0) = 2x + 5, \\ h_2 &= \mathrm{ReLU}(h_1 - 1) = \mathrm{ReLU}(x), & g_2 &= \mathrm{ReLU}(g_1 - 3\,h_1) = 2 + F_1(x), \\ h_3 &= \mathrm{ReLU}(h_2 - 1) = \mathrm{ReLU}(x - 1), & g_3 &= \mathrm{ReLU}(g_2 + 3\,h_2) = 2 + F_2(x), \end{aligned}$$

$$y = -5\,h_3 + g_3 - 2 = f(x).$$

Written out, g_2 is $2x + 5$ up to -1 and $2 - x$ after it, and g_3 equals g_2 up to 0 and is $2 + 2x$ after it. Table 1 lists the value of every neuron at $x = -2, \dots, 2$; check each entry against its rule. Figure 7 draws every neuron as a function of x, larger than the boxes of Figure 6.

x	-2	-1	0	1	2	x	-2	-1	0	1	2
h_0	0	1	2	3	4	g_0	1	1	1	1	1
h_1	0	0	1	2	3	g_1	1	3	5	7	9
h_2	0	0	0	1	2	g_2	1	3	2	1	0
h_3	0	0	0	0	1	g_3	1	3	2	4	6
y	-1	1	0	2	-1	f	-1	1	0	2	-1

Table 1: The value of every neuron of the zigzag network at $x = -2, \dots, 2$: the hinge lane on the left, the sum lane on the right, and the output y against the zigzag f. The running sum g_2 reaches 0 at $x = 2$.

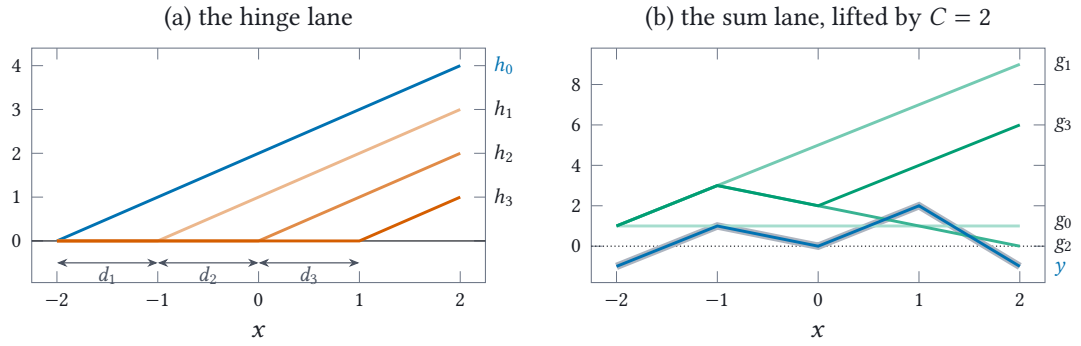


Figure 7: Every neuron of the zigzag network on $[-2, 2]$. (a) The hinge lane: $h_0 = x + 2$ (blue) and the hinges h_1, h_2, h_3, one gap apart. (b) The sum lane: the running sums $g_0, \dots, g_3$ (green, darker with depth), lifted by $C = 2$, and the output y (blue), which lies on the zigzag (grey). The running sum g_2 touches the floor at $x = 2$.

§9 Using the page

The page *Hinge-chain networks* is this network, live; it opens offline in any browser, from its file `index.html`. Its parts, from the top:

- *Targets.* Fifteen functions on $[-2, 2]$, drawn as small curves: eight piecewise linear, the zigzag first, and seven smooth. Choosing one restarts the network flat at $f(a)$, with every slope change 0 and the kinks spread evenly.
- *The plot.* The target (a thick pale line), the output y of the network (blue), the error between them (shaded), and one dashed line per kink, numbered. Drag a kink sideways to set its gap; the kinks after it move along. Above it, at the right, the mean squared error (MSE) of y on 801 points of $[-2, 2]$.
- *The network*, as in Figure 6, each knob a violet chip with a slider.
- *The buttons.* − and + remove the last layer or add one, from 1 to 10 layers, and *fit* sets every knob by the construction. The keys −, + and f do the same, and t switches between a light and a dark theme.

What fit does. It first chooses k kinks, then draws straight pieces through the target's values at a, at the kinks and at b, the *interpolant*, and sets the gaps and the slope changes by Sections 5 and 6, so that y is exactly that interpolant. The kinks are the ones, among 401 evenly spaced points of $[-2, 2]$, whose interpolant has the least squared error; a dynamic program finds them. A new layer from + goes after the last kink, in the middle of what is left of the domain, with slope change 0, so the curve stays as it was until you move its knobs or press fit. And − drops the last layer, and with it the term $\Delta_k \,\mathrm{ReLU}(x - t_k)$.

Things to try.

1. *The zigzag.* With three layers, fit sets the knobs of Figure 6, and the error is 0. Remove a layer and press fit: with two layers, fit leaves an error of about 0.12. A hinge-chain network of k layers bends only at its k kinks, so a target with K kinks needs K layers, and fit draws it exactly with K or more.
2. *The chain.* Drag kink 2 to the right and watch kink 3 move along. The four pieces keep their slopes; only the places where they bend move.
3. *A knob at zero.* Set Δ_1 to 0. Its arrow turns dashed, the curve no longer bends at -1, and every later slope goes up by 3, since each slope is the sum of the slope changes before it: $s_i = \Delta_0 + \cdots + \Delta_i$.

Smooth targets. Pick the sine, $f(x) = \sin(\pi x/2)$, press fit with one layer, then add layers one at a time, pressing fit after each. No number of layers draws the sine exactly, since it has no straight piece, but every layer buys one more kink. Figure 8 shows the fits. With one layer the fit bends once; with two its kinks sit at the peaks -1 and 1; with four it follows the sine closely. The mean squared error falls with every layer, from 0.42 with one layer to 0.023 with two, 0.0040 with four and 2.3×10^{-4} with ten, more than three orders of magnitude. This is the rate to expect: a straight piece of length h misses a curve with $|f''| \le M$ by at most $Mh^2/8$, so with $k + 1$ pieces of length $(b - a)/(k + 1)$ the error at each point is of order $1/k^2$, and the mean squared error of order $1/k^4$. The kinks of fit do better than evenly spaced ones: they leave the nearly straight middle of the sine alone and crowd where it bends most, which from four layers on roughly halves the error (Figure 8, right).

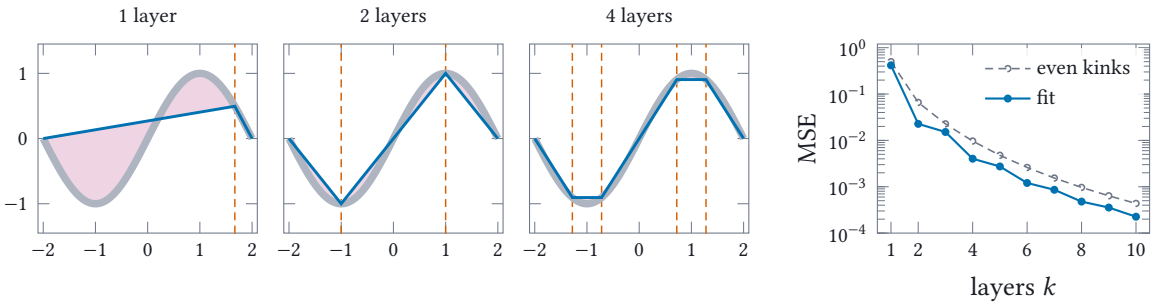


Figure 8: The sine $f(x) = \sin(\pi x/2)$ and the page's fits with 1, 2 and 4 layers: target (thick, pale), network (blue), error (shaded), kinks (dashed). Right: the mean squared error against the number of layers, with the kinks of fit and with evenly spaced kinks.

§10 Remarks

Cost. The hinge-chain network has $2k$ ReLU neurons, against $k+1$ for the single hidden layer of (1). Depth saves nothing here. What it buys is a network in which the kinks appear one layer at a time, each layer with a job that you can name.

Only on $[a,b]$. Every claim holds for $x \in [a,b]$. Outside, nothing is promised: there a running sum may be cut, and y need not continue f in any particular way. For the zigzag at $x=3$, the input of g_2 is -1 and its ReLU cuts it to 0, so $y(3)=-3$, while continuing the last piece would give -4.

Width two is the minimum. By Claim 2, one neuron per layer cannot draw an f that goes up and down, and two always suffice. This matches a result of Hanin and Sellke: for ReLU networks of any depth, width $d+1$ is the least that approximates every continuous function of d variables, and for $d=1$ that is two.[1]

§11 Summary

Take-aways

1. A continuous piecewise-linear function is fixed by its start value, its kinks and its slope changes: $f = f(a) + \sum_{i=0}^{k} \Delta_i \,\mathrm{ReLU}(x - t_i)$, one hinge per kink and hinge 0 for the first slope.
2. A hinge-chain network builds it one kink per layer, with two neurons per layer. The hinge $h_\ell = \mathrm{ReLU}(h_{\ell-1} - d_\ell)$ moves the previous kink right by the gap d_ℓ; the running sum $g_\ell = \mathrm{ReLU}(g_{\ell-1} + \Delta_{\ell-1} h_{\ell-1})$ adds the previous hinge with its slope change.
3. The running sums are ReLU neurons, so they start from a lift C, the least that keeps $F_0, \dots, F_{k-1}$ at zero or above, and the output $y = \Delta_k h_k + g_k - C$ pays it back. For the zigzag, $C=2$.
4. Only the $2k+1$ gaps and slope changes are free: the knobs of the page. Every other weight is 0 or 1, $f(a)$ comes from the target and C is computed.
5. One neuron per layer is too few, since such a network is monotone; two are enough.

§12 Exercises

Exercise 1 (One layer). Build the hinge-chain network of $f(x) = |x| - 1$ on $[-2,2]$, which has one kink. Give $f(a)$, Δ_0, Δ_1, d_1 and the least lift C, write out h_1, g_1 and y, and check that $y = f$ at $x = -2$, 0 and 2.

Exercise 2 (The tent and its lift). The tent of the page runs through $(-2,0)$, $(-1,0)$, $(0,1.5)$, $(1,0)$ and $(2,0)$.

(a) Find $f(a)$, the slope changes and the gaps. Which arrow of the network carries the weight 0?

(b) Compute the partial sums F_0, F_1, F_2 and the least lift. Which running sum touches the floor, and where?

(c) Take $C=1$ instead. Where is the network wrong, and by how much at $x=2$?

Exercise 3 (A negative gap). (a) Show that for $d<0$, $\mathrm{ReLU}\big(\mathrm{ReLU}(u) - d\big) = \mathrm{ReLU}(u) - d$ for every u: the ReLU never cuts. For which u does it differ from $\mathrm{ReLU}(u-d)$?

(b) In the zigzag network, set $d_2 = -0.5$, so that $t_2 = t_1 + d_2 = -1.5$. Compute h_2 at $x = -2, \dots, 2$ and compare with $\mathrm{ReLU}(x - t_2)$. Is h_2 a hinge at t_2?

(c) Why does the page keep every gap at zero or above?

[1] B. Hanin and M. Sellke, *Approximating continuous functions by ReLU nets of minimal width*, arXiv:1710.11278, 2017.

Exercise 4 (Width one). (a) Prove Claim 2.

(b) Show more: the output of a network with one neuron per hidden layer is constant up to some point, then linear, then constant again (each part may be empty). How many kinks can it have?

Appendix A Proofs

Proof of Claim 1. Call the right-hand side of (1) $R(x)$. On the piece $[t_j, t_{j+1}]$, the hinges with $i \le j$ are linear, since $x - t_i \ge 0$ there, and the hinges with $i > j$ vanish, since $x \le t_{j+1} \le t_i$. So R is affine on the piece, with slope $\Delta_0 + \Delta_1 + \cdots + \Delta_j = s_0 + (s_1 - s_0) + \cdots + (s_j - s_{j-1}) = s_j$, the slope of f. At $x = a$ every hinge vanishes and $R(a) = f(a)$. Two continuous functions that agree at a and have the same slope on $[a, t_1]$ agree on $[a, t_1]$, hence at t_1, hence on $[t_1, t_2]$, and so on, piece by piece up to b. □

The same argument describes the partial sums. If $x \le t_{\ell+1}$, every hinge with $i \ge \ell + 1$ vanishes, so $F_\ell(x) = F_k(x) = f(x)$. If $x \ge t_\ell$, every hinge in F_ℓ is linear, so F_ℓ is affine there, with slope $\Delta_0 + \cdots + \Delta_\ell = s_\ell$. So F_ℓ is f up to $t_{\ell+1}$ and goes on straight after it with the slope s_ℓ, as Section 3 states.

Claim 2 is part (a) of Exercise 4; its proof is in Appendix B.

Proof of Claim 3. If $u \ge 0$, then $\mathrm{ReLU}(u) = u$ and the two sides are the same expression. If $u < 0$, then $\mathrm{ReLU}(u) - d = -d \le 0$, because $d \ge 0$, and $u - d < 0$, so both sides are 0. □

Proof of Claim 4. For $\ell = 0$, $h_0 = x - a = \mathrm{ReLU}(x - t_0)$ because $x \ge a$. If $h_{\ell-1} = \mathrm{ReLU}(x - t_{\ell-1})$, then (3) and Claim 3 with $u = x - t_{\ell-1}$ and $d = d_\ell \ge 0$ give

$$h_\ell = \mathrm{ReLU}\big(\mathrm{ReLU}(x - t_{\ell-1}) - d_\ell\big) = \mathrm{ReLU}(x - t_{\ell-1} - d_\ell) = \mathrm{ReLU}(x - t_\ell).$$ □

Proof of Claim 5. The hinges are Claim 4. For the running sums, $g_0 = C + f(a) = C + F_{-1}$. Let $\ell \ge 1$ and assume $g_{\ell-1} = C + F_{\ell-2}$. By Claim 4, the input of the ReLU of g_ℓ is

$$g_{\ell-1} + \Delta_{\ell-1} h_{\ell-1} = C + F_{\ell-2}(x) + \Delta_{\ell-1}\,\mathrm{ReLU}(x - t_{\ell-1}) = C + F_{\ell-1}(x),$$

which is at least 0 by (6), since $\ell - 1 \le k - 1$. So the ReLU passes it on unchanged, and $g_\ell = C + F_{\ell-1}$. Finally, $y = \Delta_k\,\mathrm{ReLU}(x - t_k) + C + F_{k-1}(x) - C = F_k(x) = f(x)$ by Claim 1. □

The least lift. Each F_i is continuous and piecewise linear on $[a, b]$, with its kinks among $t_1, \ldots, t_i$, and a linear function on an interval is lowest at one of its ends. So the lowest value of F_i on $[a, b]$ sits at a, at b or at a kink. Condition (6) says $C \ge -\min_x F_i(x)$ for $i = 0, \ldots, k-1$; together with $C \ge 0$ this gives C_*.

Proof of Claim 6. Fix $x \in [a, b]$ and any C. Write $u_\ell = C + F_{\ell-1}(x)$ for the value that g_ℓ should have, and $e_\ell = g_\ell(x) - u_\ell$ for its error, so that $e_0 = 0$. The hinge lane never reads the sum lane, so Claim 4 holds whatever C is, and the input of the ReLU of g_ℓ is

$$g_{\ell-1} + \Delta_{\ell-1} h_{\ell-1} = u_{\ell-1} + e_{\ell-1} + \Delta_{\ell-1}\,\mathrm{ReLU}(x - t_{\ell-1}) = u_\ell + e_{\ell-1}.$$

Since $\mathrm{ReLU}(v) \ge v$, we get $e_\ell \ge e_{\ell-1}$: the error never decreases, and $e_\ell \ge 0$ for every ℓ. Now let ℓ be a layer with $u_\ell < 0$, which exists when (6) fails at x. If $u_\ell + e_{\ell-1} < 0$, the ReLU cuts, $g_\ell = 0$ and $e_\ell = -u_\ell > 0$. Otherwise $e_{\ell-1} \ge -u_\ell > 0$, and $e_\ell \ge e_{\ell-1} > 0$. Either way $e_\ell > 0$, hence $e_k > 0$, and

$$y(x) = \Delta_k\,\mathrm{ReLU}(x - t_k) + u_k + e_k - C = F_k(x) + e_k = f(x) + e_k > f(x).$$ □

Appendix B Solutions to the exercises

Exercise 1. The function starts at $f(a) = f(-2) = 1$ and has its kink at $t_1 = 0$, with slopes $s_0 = -1$ and $s_1 = 1$. So $\Delta_0 = -1$, $\Delta_1 = 2$ and $d_1 = 0 - (-2) = 2$. The only partial sum that passes through a running-sum neuron is $F_0 = 1 - (x + 2) = -x - 1$, which is lowest at $x = 2$, where it is -3; so $C = 3$ and $g_0 = C + f(a) = 4$. Then $h_1 = \mathrm{ReLU}(x + 2 - 2) = \mathrm{ReLU}(x)$, $g_1 = \mathrm{ReLU}\big(4 - (x + 2)\big) = 2 - x$, which touches the floor at $x = 2$, and

$$y = 2h_1 + g_1 - 3 = 2\,\mathrm{ReLU}(x) - x - 1 = |x| - 1,$$

since $2\,\mathrm{ReLU}(x) - x = |x|$. At $x = -2, 0, 2$ the output is $0 + 4 - 3 = 1$, $0 + 2 - 3 = -1$ and $4 + 0 - 3 = 1$, the values of $|x| - 1$.

Exercise 2. (a) $f(a) = 0$, the slopes are $(0, 1.5, -1.5, 0)$, so $\Delta = (0, 1.5, -3, 1.5)$, and $d = (1, 1, 1)$. The arrow from h_0 into g_1 carries $\Delta_0 = 0$, since the first piece is flat; on the page it is dashed.

(b) $F_0 = 0$ and $F_1 = 1.5\,\mathrm{ReLU}(x + 1)$ never go below zero. $F_2 = F_1 - 3\,\mathrm{ReLU}(x)$ is the tent up to 1 and goes on with the slope -1.5 of the third piece after it, $F_2 = 1.5 - 1.5x$ for $x \geq 0$, down to -1.5 at $x = 2$. So $C = 1.5$, set by F_2 at the right end: the running sum $g_3 = C + F_2$ touches the floor at $x = 2$.

(c) With $C = 1$, the input of g_3 is $1 + F_2(x) = 2.5 - 1.5x$ for $x \geq 0$, which is negative for $x > 5/3$. There g_3 is cut to 0 and, by Claim 6, $y > f$; in fact $y = 1.5(x - 1) + 0 - 1 = 1.5x - 2.5$, while $f = 0$. The error is $1.5x - 2.5$, which is 0.5 at $x = 2$. Left of $5/3$ the network is exact.

Exercise 3. (a) For $d < 0$, $\mathrm{ReLU}(u) - d = \mathrm{ReLU}(u) + |d| > 0$, so the outer ReLU passes it on: $\mathrm{ReLU}\big(\mathrm{ReLU}(u) - d\big) = \mathrm{ReLU}(u) + |d|$. This is the hinge $\mathrm{ReLU}(u)$, raised by $|d|$, with its kink still at $u = 0$. The function $\mathrm{ReLU}(u - d) = \mathrm{ReLU}(u + |d|)$ has its kink at $u = -|d|$. The two agree for $u \geq 0$, where both are $u + |d|$, and differ for every $u < 0$, where the first is $|d|$ and the second is less.

(b) $h_1 = \mathrm{ReLU}(x+1)$ is $0, 0, 1, 2, 3$ at $x = -2, \ldots, 2$, and $h_2 = h_1 + 0.5$ is $0.5, 0.5, 1.5, 2.5, 3.5$. But $\mathrm{ReLU}(x+1.5)$ is $0, 0.5, 1.5, 2.5, 3.5$, so the two differ at $x = -2$, and in fact on all of $[-2, -1)$. The neuron h_2 is the hinge at -1 raised by 0.5; its kink stays at $t_1 = -1$, and there is no kink at $t_2 = -1.5$.

(c) Only for $d_\ell \geq 0$ is a lowered and cut hinge a moved hinge (Claim 3). With every gap at zero or above, the kinks $t_\ell = t_{\ell-1} + d_\ell$ also stay in order.

Exercise 4. (a) If $w \geq 0$, the map $u \mapsto wu + c$ is non-decreasing, and so is $u \mapsto \mathrm{ReLU}(wu + c)$, because the ReLU is non-decreasing. If $w \leq 0$, both are non-increasing. So every hidden layer is a monotone map of its input, and so is the affine output. A composition of monotone maps is monotone: two non-decreasing maps, or two non-increasing ones, compose to a non-decreasing map, and one of each to a non-increasing map. So the network computes a monotone function of x, which cannot rise and then fall, as the zigzag does.

(b) Call a function of x a *ramp* if, for some $p \leq q$ in $[-\infty, \infty]$, it is constant on $(-\infty, p]$, linear on $[p, q]$ and constant on $[q, \infty)$. The input x is a ramp, and so is every affine function of a ramp. Let v be a ramp, say non-decreasing (the other case is the mirror image). If $v \geq 0$ everywhere, $\mathrm{ReLU}(v) = v$; if $v \leq 0$ everywhere, $\mathrm{ReLU}(v) = 0$; otherwise v crosses zero at one point r of $[p, q]$, and $\mathrm{ReLU}(v)$ is 0 up to r, linear on $[r, q]$ and constant after q. In every case $\mathrm{ReLU}(v)$ is a ramp. By induction over the layers, every neuron and the output are ramps, and a ramp has at most two kinks, at p and at q.